Unit 14 - Event Driven Programming
Unit 15 - Object Oriented Programming
Assignment 2 - Theory

Introduction

Scenario / Vocational Context - *As a junior, though vital, member of your organisations' software development team you are required to contribute to documentation efforts concerning best practices in software development. Your line manager, the Head of Software Development, has asked you to answer a number of questions.*

Introduction

In this document, I will be explaining both even-driven and object oriented programming paradigms, discussing the key features and the importance that each programming language has on certain applications and situations.

In this theory, I will mostly be mentioning both programming language paradigms, however there will be a limited amount of mentioning of specific programming languages, such as C# and small amount of Java, for comparison purposes or for ease of explanation.

Task 1

(U14:P1) - Explain the key features of event-driven programs

Task description - *By answering the following questions you will explain the key features of event-driven programs. Please use the question as a subheading. Each of the following requires only a relatively short answer. Provide a good example wherever possible.*

What is an event handler?

Event handlers, in event driven programming, are implemented in order to callback a procedure, usually independently of other processes, to designated events triggered through inputs received from a user. For example, the C# programming language event model is based on a type of "publish-subscribe" pattern. Which is essentially when a class, or publisher, triggers an event, causing the other class, or subscriber, to receive that event. The event handler in this situation is the subscriber that contains the code which handles specific events.

Usually, event handlers are small sections that are most likely implemented on a part of a larger program, such as an operating system or video game application, in order to add simple but effective functions to a program/system. Hence the fact that event handlers are seen as self-contained, which is intended for separation and ease of the programming language. This is so the programmer can read through the code easier and possibly debug any errors. Which

may also be why event handlers may be also known as subroutines, due to the fact they are small parts of code implemented into a larger program.

How are event handlers triggered?

As mentioned previously, event handlers are triggered through inputs received from a user, which is called an event. The inputs from a user's interaction are usually found with the GUI control, such as mouse activity, keystrokes, action selections or even timer expirations. Essentially, event handlers are segments of code which is run when the event occurs, such as the user interacting with the mouse and keyboard on a button for example.

Provide a simple example of an event handler (with simple code).

```
private void btnCreateReptile_Click(object sender, EventArgs e)
{
    // Create a new instance of a Reptile. Call it fred
    Animal fred = new Reptile();

    string noise = fred.Speak();
    string about = fred.About();
    txtAbout.Text = noise + ". " + about;

    // And add fred to the list of animals
    animals.Add(fred);
    pos += 1;
    lblPos.Text = pos.ToString();
    lblAnimalCount.Text = "You have " + animals.Count.ToString() + " animals in your collection";
}
```

On the screenshot above, it shows an eventhandler for a button from the "AnimalKingdom" application. What this screenshot shows is the eventhandler, which is in the form of a button on the user interface. Where it would create a new instance of the class "Reptile" when the user presses the said button.

With the creation of this new "Reptile" class, it would trigger the "Speak()" and "About()" methods, which would carry out the function of these methods. Also, giving the user a message that the class has been added to the "collection", which is a list of classes that the user can add classes to and read them.

List three possible actions that could trigger the above event handler? (eg mouse click)

There are multiple examples of event handlers to trigger the event shown in the screenshot shown above. Events such as the input from keys on the keyboard or the interaction of the mouse, such as pressing the mouse buttons or hovering the cursor over a the event.

One example of an action that could trigger the event handler, which is shown in the example screenshot above, is through the click event. For instance, the event to trigger the eventhandler

is a click event, meaning that the user would have the click the button via mouse interaction, in order to create the instance of the class.

Another example of a possible trigger would be a single keystroke on a keyboard, through the “KeyDown”,“KeyUp” or “KeyPress” events. Essentially, a KeyDown would be triggered when a designated key on the keyboard is pressed downwards, while “KeyUp” would be when a key is released upon pressing and “KeyPress” would be when the key is pressed in general. Basically, a keystroke from a designated key, such as the “Enter” key for example, would trigger the application to carry out the eventhandler due to input of the event.

The final example is through a mouse hover event, which is essentially mouse/trackpad movements of a user over the button. Which basically triggers the eventhandler when the user hovers the mouse cursor over the button, triggering the “MouseHover” event. Causing the same eventhandler to get triggered as the last examples.

Give an example of a service oriented event-driven program, and an event that it would respond to.

A service oriented program is a style of designing applications in such a way so they are made up of software agents, which wouldn’t be seen by the user. Usually, a service oriented program has a simple and well defined interface, meaning that it wouldn’t have an effect on the processing power of the PC.

To put it simply, service oriented is essentially a background application which is not necessarily seen by the user. An example of a service oriented program would be when a user inserts an external storage drive, such as a USB flash drive. Which would run a service within the operating system in order to install the necessary drivers needed to access the drive to the computer system.

Provide an example of a time driven event-driven program, and an event that it would respond to.

Time driven is essentially a program which uses timed events in order to execute a series of code. The code would specifically be run on a timely basis, such as once an hour, week or month. Meaning that a time trigger would have to be preset automatically or by the user, in order to initiate the trigger to carry out a task.

For instance, a timed installation for a Windows update can be set up to check for updates on the desired time span, such as one a month. Which it would then send a notification to the user if there has been a WIndows update installed or ready to install. This can also be used within antivirus programmes to set for a virus scan on an allocated time span.

How do you make an event occur at regular time intervals

In order to make a timed even occur, a timer class needs to be used to create accurate time intervals between these events. A timer class is essentially a class which generates a specific event after a set time interval, usually without a secondary thread. Essentially, if a specific code needs to be executed with a timer class, after a certain interval of time has passed, the timer control will be used.

What might happen if one event triggered another event which then triggered the first event

This will cause what is called a Stack Overflow. Which is essentially an unpleasant condition which a computer program will try to use as much memory space as possible, usually resulting to more memory space than the call stack has available. This type of condition is very undesirable to have due to the problems it could cause to the PC, so it should be avoided at all times.

What three key features of MS Visual Studio ease the development of event driven programs that have a Graphical User Interface (GUI)?

Microsoft's Visual Studio has a GUI to help the programmer to add graphical components to their applications, to help in the development of an application. These features ease the development due to the fact they give a visual representation of the different sections of the application. There are multiple features in Visual Studio that provide a GUI for the programmer.

For example, one feature of a GUI in MS Visual Studio is the form design view. Which is essentially the design of the User Interface for the application that is currently being designed. In this form design view, the user can select a variety of tools from the tool box such as a button or text box for example. Giving the programmer the ability to create a graphical user interface for the application itself, which requires a form of GUI in order to create it effectively.

Another feature of a GUI in Visual Studio is class diagram. Which is basically a GUI which the programmer can add classes in the form of boxes, giving them the ability to add properties such as methods and fields for example. These classes, represented as boxes in the GUI, can be moved around by the programmer and be inherited with others, creating a hierarchy of derived classes.

Yet another feature would be the thread call stacks in a single graphical view. Which is essentially a GUI, similar to the class diagram, which the programmer can see the expressions across multiple threads, in order to compare the values of each of them. The programmer can also switch between execution context, using the thread markers to identify which lines of code the said threads are executing live.

Essentially, GUIs are very useful tools in MS Visual Studio for the ease of development. Due to the fact it gives the programmer the ability to manipulate the program visually, instead of

inputting data and click different things. Which is advantageous in creating a an application, whether it is with the hierarchy of functions within the application or creating a UI, making it a lot easier to develop an application.

Task 2

(U14:M1) - Discuss how an operating system can be viewed as an event-driven application

Task description - *Using two good examples, explain clearly how events are triggered and handled by an operating system, thereby demonstrating that an operating system is an example of an event driven program.*

Discuss how an operating system can be viewed as an event-driven application

Generally speaking, operating systems can be viewed as event-driven applications as a whole. Since the operating system completely relies on the uses of triggers for events, in order to make the operating system carry out the said events.

In order for the operating system to actually register the user's input, the interaction of peripherals is essential, such as a keyboard and a mouse. The reason being that all the features found on the operating system application needs the user's interaction in order to actually carry out a function.

Essentially, the reason why operating systems require user interaction in order for it to function, is the fact that operating systems work purely through triggers. Meaning that the user would control when everything is done, when applications are launched, when to rename a file or when to delete/uninstall an application, file or folder.

Meaning that every time the user would want something done, they would need to make it happen by interacting with the application, which could be opening it with a mouse click or deleting through a menu. Therefore meaning that the user has total control over the computer and the actions that are carried out.

To support this allegation of the operating system being completely (or mostly) event driven, there are multiple examples within the operating system that prove it is an event driven application.

For example, if the user decides when to log out of the computer, when to shut down the computer or even when to switch accounts. The user would have to select the shutdown eventhandler from start menu on windows or by clicking the apple logo in the top left of the screen on MacOS. Which would, upon being pressed on by the user, would trigger the event of the PC cancelling all power to it's components, resulting in a shut down.

Therefore, the eventhandler for this situation would the Windows/Apple button, while the event is the computer shutting down. Essentially, the event in this instance is the user clicking the Shutdown button and the reaction from the operating system is the computer going off.

Another example of an event driven feature on an operating system is the creation/renaming of a folder. Essentially, in order to create or even rename folders, the user would need to use the right click event, pressing the rename eventhandler (in the form of a button), which the user would then input into the text box what they desire, with the folder name being changed upon the user's input from the keyboard events.

The events carried out in this example would have been the users interaction with the mouse to right click the folder, then interacting with a button that opens a text box. Which the user would input the keystrokes from the keyboard in order to create a name for a folder.

This also applies when the user wants to create a new folder and wants to input a new name for the said folder. Since the operating system cannot do it by itself, it needs the user's interaction in order to create a new, named folder. Meaning that the interaction of the mouse and the input of the keyboard being the essential event driven task, in order for the operating system to rename or create a folder.

Another good example of an event driven task in an OS is accessing the file explorer, that is only accessed through a "MouseClick" event. Meaning when a user clicks on the button for the file explorer using the mouse, it comes up with a window and a larger variety of other event handlers, in the form of files, folders and documents which can be interacted with by the user, also utilising eventhanders to trigger events.

The possible events that have occurred in this example was when the user mouse clicked on the button event handler, causing the event to occur, which would be the extended list of buttons opening up in the user interface, indicated as the start menu.

Yet another good example is the fact that there are drag and drop characteristics found on areas of the operating system where it features files and folders, such as the desktop and the file explorer. For instance, the most common place to see the Recycle Bin on a computer desktop is the top left hand corner, granted it hasn't been moved since the computer was set up.

Which can be moved away when the user interacts with the mouse in order to click the left mouse button, hold the left mouse button and move the mouse to adhere to the desired location in which the user wants the file to be located. This is also possible with all the other icons, such as "Internet Explorer" and "My Documents" for example.

The reason that this can be seen as event driven is the fact that the drag and drop feature also requires the user to use the mouse click and hold to move different files/folders/documents inside another folder. For example, if the user wants to insert a word document into a folder, which has a large chance of already containing other similar files and documents. The user would have to use the mouse interaction in order to carry this out. Which is done by clicking and hold the left mouse button, then moving the icon into the desired folder.

Granted that they don't have the same file name or the operating system would require the user to change it, in order to not get confused with the multiple files. The event driven action in these would be the icon being clicked on and held with the left mouse button and dragged with the user's movement of the mouse. Causing the visual reaction of the user to be the object, such as the file or folder, being dragged into a different folder to where it previously was.

Also, while the movement of files and folders are event driven, there are also event handlers that causes an event that changes to the visuals of a file, folder or button. For instance, when the user moves the mouse cursor over the icons without clicking on it, it changes the icon. Causing what seems a visual illumination on Microsoft's Windows button or the MacOS apple logo. When it is actually the icon changing image upon recognition of the user's mouse over the icon.

The event in this example is the user hovering the mouse cursor over the icon, with the event handler being changing the icon changing to a different image upon said event, giving the illusion of being illuminated.

These are only a few of many examples of event driven functions within an operating system. Since an operating system essentially required the user's interaction to carry out a task, there could be many other examples of how the operating software can viewed as event driven.

So, with this evidence, the fact that the operating system needs user interaction is vital for the application to carry out its intended purpose. To elaborate on this, if a user wants to open an application, they would have to click on the application that they want to open, causing it to launch. Since an operating system relies these type of events, it is important that it has the fundamentals of event-driven programming is needed in order to carry out the said events.

For example, if there wasn't any allocated event handlers in the operating system, it wouldn't be able to load any of the of the applications or other functions on the computer's operating system. Which would render the operating system unless for user, as the computer wouldn't know what applications the user will be accessing, which would either make the operating system do absolutely nothing or randomly accessing different applications. Hence why it must contain event handlers, so the operating system can deal with the user's interaction.

However, bad development of an operating system gives it the possibility of the operating system going through a stack overflow. Which, as mentioned previously, is a very undesirable

condition. Which can cause the entire PC to crash, since the operating system is a vital application in a computer.

In conclusion, the peripherals such as a mouse and keyboard are essential to carry out events, which the event handlers in the operating system would respond to. For example, a mouse uses movement which is detected through a sensor, the left for further interaction of applications, right click for the selection the options of an applications and the scroller which can be used to navigate up or down the applications found in an operating system with the features to do so.

Essentially, these are all done through events linked to the user's interaction with the mouse. If the operating system wasn't implemented with such event driven functions using triggers, the user wouldn't be able to use peripherals such as the mouse as it wouldn't be able to process the user interaction, therefore cancelling the important use of the mouse for navigation.

Meaning that the keyboard must also be event driven, otherwise none of the keys would be available for interaction, making typing completely impossible. The reason for this is that, every time a key is pressed, the computer registers the information then passes it through to the program that you are using to allow the letters to be virtually displayed onto the monitor.

The way that this is done, it is registered as either a KeyDown (Occurs when a key is pressed via user), KeyPress (Occurs when a character, space or backspace key is pressed via user) or KeyUp (Occurs when a key is released via user) command. Meaning that this would be linked with other data that is allocated with the KeyDown/KeyPress/KeyUp commands, such as the start menu opening when the Windows key has pressed by the user.

Hence why peripherals, such as a keyboard and mouse, must have event driven code written in order for it to function on an operating system. Which can be also said for the mouse interaction, since it requires user interaction in order for it to function accordingly. Making it a significant reason why operating systems are seen as event driven applications, since it is vital to do so.

In retrospect, the peripherals that the user interacts with is essential in order to use an operating system, since it requires user interaction. Therefore, the entire operating system, which is essentially a GUI for user interaction, can be seen as event driven application. The reason for this fact is that the operating system cannot do anything, such as opening a seperate application, without the user's interaction.

Task 3

(U14:D1) - Evaluate the suitability of event-driven programs for non-graphical applications

Task description - *An event-driven program does not have to have a Graphical User Interface (GUI). Describe, in detail, a good example of an event-driven program/application that does not*

have a GUI. Explain clearly whether event-driven programming is, or is not, suitable for non-GUI programs/applications.

Evaluate the suitability of event-driven programs for non-graphical applications

Even though even driven programs are mostly used for application with GUI, there are some event driven programs that can be limited to their graphical user interface. Whereas some applications may have complex user interfaces for the user to interact with, others may be plainly text based.

However, the latter may be vastly more powerful and useful compared to graphically integrated applications, which can certainly be a lot faster than applications with a graphical user interface. Due to the fact it is less demanding of the computer's hardware, which may be problematic for low-specification systems.

Typically, applications with a graphical interface may seem easier to use and more welcoming when compared to the less pleasant looking text based applications, especially if the graphics are well designed.

However, this can sometimes lead to text based applications being under appreciated, even if it is widely superior to applications with GUIs on what the user is trying to accomplish. For instance, if the user has a system that's designed to monitor and respond to random alerts, such as a data center monitoring application. Then it is possible that event driven programming can be used, to a significantly useful extent, without the need of a GUI to carry out the task.

The reason that non-GUI applications might be used, is due to the fact that non-GUI applications don't demand much memory to be run, as it is simple enough for any computer to process. Which is useful for low powered computers, that may only be used or processing simple applications, usually found in places of work by employers. Basically, not all applications made for PC's have a graphical user interfaces.

Furthermore, there are examples non-graphical applications applied in everyday life which aren't restricted only to personal computers. For instance, a thermostat used to alter the temperature of a room could be implemented within a simplified system, such as a mobile device or PDA, with the help of Wi-Fi connections. Making it possible for it to be a Non-GUI event driven application, since it only uses a couple of LEDs to display what the temperature is and whether to change it, which can be easily done without a GUI, only with the perception of text through the LEDs.

Also, there has been a surge implementation of computer system in modern cars, therefore integrating programming into the software of the cars. The systems are usually responsible for the electronics in a car which displays the amount of fuel, the speed of the car, the RPM of the engine, the miles clocked, the temperature outside, the current time and what radio station is

playing. All of which is displayed via LEDs, driven by the data provided by the events that occur due to both the user and the cer's input of the information necessary.

Essentially, the majority of non-graphical applications are driven by in-direct commands and can't all be directly controlled by a user, but all of this is dependent on the purpose. An example which may be applicable for comparison would be a microwave for instance, which completely relies on the input of the user. The user can interact with the microwave, and a whole range of different options are available. Meaning that there are some pre-sets that can be applied, so that the user can initiate the intended purpose of the machine with insignificant interaction.

Another non-GUI application found in a computer or server would be applications which monitor stock prices, without the need for the user to interact with the application. Since the stocks are calculating the total income from purchase/selling of products, calculating the profit margins of the company. With the events being triggered by the input of information being given at life time, by the means of a loop.

Which would require a constant internet connection in order to obtain the live feed of information. However, while non-GUI applications can prove to be useful, there are also non-GUI applications where event driven programming is either inappropriate or less superior to other methods of programming.

An example for this is the use of an Artificial Intelligence, only if it is not self learning however, in an application, such as a video game, with the purpose of carrying out tasks independently. Meaning that the actions and responses of this AI are all sequential, mostly requiring independent interaction with the application's environment.

Due to the nature of the event driven programming paradigm, this would be impossible since it requires some sort of interaction from another form of intelligent interaction, such as a user's interaction, in order for the events to actually trigger and carry out a function. In this case, choosing event driven programming for the main portion of the application would be unwise.

With the exception of the previous scenarios, using an event driven programming paradigm wouldn't be a valid option for a non-learning AI system. However, it doesn't mean that it isn't good choice for non-GUI applications as a whole. With applications that require the interaction of a user in order to change an application's information or behaviour, the event driven programming paradigm would be an outstanding choice compared to other paradigms.

In conclusion, I believe that event driven languages can be suitable for non-graphical applications, due to the fact that some of them require user interaction, with some being essential in everyday life. Due to this fact, event driven programming can be very effective and helpful.

Sometimes, non-graphical applications can greatly underappreciated for their role, especially in the background, away of what the user sees in an operating system. However, it is understandable mainly due to the fact that they don't have a noticeable graphical interface that the user can fully interact with.

Also due to the fact that event driven focuses on the user's interaction in order for event driven to work much more effectively in terms of interacting with the application. To elaborate, non-graphical applications don't utilise the usefulness of graphical applications for user interaction, which event driven programming excels on.

So, while these non-graphical applications may be somewhat important, since they provide us with simplicity and still contain user interaction not only in computer systems, but in most electronics used in the modern day.

Overall, even though it is possible to have non-graphical applications as event driven programs. The use of event driven application on a graphical interface can prove to be of much higher importance for this type programming archetype. Simply due to the fact that event driven programming is very beneficial for a user to interact with an application that has a GUI, rather than one that doesn't have a GUI.

Task 4

(U15:P1) - Explain the key features of object oriented programs

Task description - *By answering the following questions you will explain the key features of object-oriented programs. Please use the question as a subheading. Each of the following requires only a relatively short answer. Provide a good example wherever possible.*

What is a method?

A method is essentially a subroutine, often referred as a procedure, associated with an object class which are always used when defining a class or a object. Typically, it will define the behavior that is used by objects methods also have access to data within the classes, which is called binding.

What a method essentially does is it controls the object class, which is always made up of data and it's behaviour on the application. Therefore, using methods will allow the control of the objects within a program by accessing the data which is found with said object. Which would potentially make the application easier and faster to develop, due the improvement of data integrity within an application.

Explain how you would pass information to a method

In order to pass information to a method, a parameter would have to be used within the code. Which is essentially a variable that is used to pass information between methods and even functions. This information that would be passed with the parameter variable may be referred as an argument.

The way in which these arguments are passed to methods usually depends on the type of programming language and system. Usually, the rules associated with a parameter is what order the arguments should be and whether arguments will be passed by value, reference or other functions, depending on the system/programming language

What is a class?

A class is a name and a description something is given to an object, usually found with methods to determine the behaviour of the object. It is most commonly used to put objects into groups, which will allow the programmer to have classifications of similar objects, rather than just having to change them one by one.

Classes make a lot easier to organise the program code and helps the programmer keep track of everything, without constantly reading through the program code. Which would be extremely time consuming, especially on a larger application. Essentially, a class is a collection of objects of similar type. Once a class is defined, any number of objects can be created which belong to that class.

What is an instance?

An instance is essentially a specific occurance of any object, which is usually found during the runtime of a computer program. Essentially, an instance can be similar to an “object”, since they both are a particular value, or realisation. Which leads to the creation of a realised instance, called an instantiation.

In this case, they would be called an instance object, with the instance emphasising the distinct identity of the said object. Meaning that no matter what each realised variation of that object is, it is still considered an instance.

Explain one advantage of storing a class in its own file/module.

One advantage of creating a class in it’s own file/module, is that it can be used on multiple applications, without the code being copied multiple times. Meaning that one class, containing a certain amount of objects, can be used on multiple applications without affecting the integrity of data. Essentially, it allows the programmer to make use of previously written code in order to expand on an application’s capabilities.

Name two ways of storing references to an object instance?

One way of storing references to an object, is though a variable. Which is essentially like a contained for various amounts and different types of information, in turn making it easier to reference the information within a program. For instance, instead of typing the same code repeatedly, the programmer could make a variable in the desired name to store the functions needed, and use it whenever the programmer wants to reference it.

Another way of storing references to an object instance is through a list, otherwise known as a sequence or an array. Which is essentially a method of storing a multiple number of variables that have the ability of occuring more than one time. This is done by grouping all the variables under a single name. Granting the ability for the programmer to define a single variable, accessing its members, instead of writing a program with many variables. This essentially allows the programmer to deal with large data sets using a small line of code.

Suppose you were creating an OO application for storing details on various animals (such as the AnimalKingdom example we have studied). Why, in this case, would inheritance be useful? Use suitable technical language.

When creating an application which stores a hierarchy of classified objects, inheritance would be a very useful object oriented technique. The reason being that it helps reduce the amount of code written for the class diagram, since I would need to write less code and it make the whole process much easier.

Also, inheritance helps in the creation of a suitable hierarchy for the appropriate classes. For example, there are multiple animals that fall under the vertebrate class. Also, since all vertebrates has has the same base set of traits, such as having an internal backbone. It makes it much easier if every vertebrate inherits the traits of the vertebrate class, while still keeping unique ones.

What is an abstract class?

An abstract class is a generic class or type of object that can never be instantiated in the program, requiring subclases to provide implementation for the abstract methods. Abstract classes always would contain one or more abstract methods, which is essentially a method that is declared, but never contains the availability of any implementation.

Name three examples of derived classes (from the AnimalKingdom example)?

A derived class is essentially a class which inherits the variables from an already existing class, called a base class. The derived class would usually be empty or have very little methods in it due to the fact it is already borrowing all the methods from the base class. An example of this would be from the animal kingdom, which has a hierarchy of different types of animals in a class.

For example, the “Bird” class is a derived class from the “Vertebrate” abstract class, making the “Vertebrate” abstract class the base class and the “Bird” class the derived class. With the “Bird” class also containing a method of its own called “Bird”.

Another example would be the “Amphibian” class, which is also considered a derived class since it inherits the methods from the “Vertebrate” abstract class, in a similar fashion to the “Bird” class. Since it inherits the “Speak()” method but executes it differently, with the only exception being that it contains its own method called “Amphibian”.

Lastly, the “Human” class is a completely derived class, since it doesn’t feature methods of its own, instead inheriting all the methods within the “Mammal” class. Those methods being the “About()” method, “Mammal” method and the “About()” method, using all of these in a different fashion to the rest of the subclasses found under mammal.

Overall, all the derived classes, such as the “Bird”, “Arachnid” and “Human” class is considered a derived class. Due to the fact it inherits the “Speak()” method from the “Vertebrate”, ”Invertebrate” and “Mammal” class. Meaning that the derived classes uses the same “Speak()” method as the base class, but uses them in a different manner to said base classes.

Show how an abstract class and a derived class can share method names (with reference to polymorphism)?

For an abstract class to share method names to a derived class, it would have to inherit the methods through the means of inheritance. However, if only inheritance was used to share the same method names, the program will get confused from the duplication and it would cause an error. Hence why polymorphism is important for aspects similar to this, the reason being is due to the fact it can either use overriding or overloading techniques.

Overriding is essentially a way of a subclass being able to provide a completely different execution for a method, that is already been previously defined and/or implemented in a base class, or one of the many bass classes. This gives the derived class the ability to have similar names, signature and even parameters as the parent class, in this case the abstract class, but executed in a different use for it when compared to other classes also inheriting from the abstract class.

Overloading is essentially the ability of a single derived class to use a wide array of methods, from an abstract class, that differ in the input and output parameters. This gives the derived class the ability to execute the same task as the other classes inherited with the abstract class, but with slightly different sets of parameters.

Essentially, polymorphism helps by sharing method names through overriding and overloading. WIth overriding enabling the subclass to handle different types of different data with a similar

interface, in this case from an abstract class. While overloading helps to avoid code which are prone to errors, by having the same method name used multiple times but with different set of parameters.

Encapsulation means the "the dog, the whole dog, and nothing but the dog" – explain.

The phrase “the dog, the whole dog, and nothing but the dog” means that, the class created through encapsulation, would have all of it’s functions only working for said class. Essentially, this means that the class created through encapsulation would only have the functions working for the class alone, without being inherited to other classes, hence “the whole dog and nothing but the dog”.

Encapsulation requires the hiding of (some) member data (properties), and providing access via Get() and Set() methods. Briefly explain an advantage of providing access only via these ‘getters’ and ‘setters’.

The advantage of only providing access through the said “getters” and “setters” is to essentially keep the data encapsulated from the rest of the code, which is essentially keeping the properties only the the individual class. So, if the code can only be referenced to the individual class, there could be another property with an identical name and the program wouldn’t be confused with the similarly named properties.

Why might you need more than one abstract class in an application?

The reason that more than one abstract class might be used more than once is due to the fact abstract classes are useful when creating hierarchies of classes. Due to the fact they make it possible to specify an never changing level of functionality in some methods. Essentially, abstract classes are used as a basis for creating specific objects that is similar to its purpose, classification or the set of operations it supports.

Task 5

(U15:M1) - explain the importance of encapsulation, polymorphism and inheritance on object oriented programming

Task description - *Using an example from program code that you have written, show clearly how encapsulation, polymorphism and inheritance have made a vital contribution to the creation of an application. You can help yourself in answering this question by demonstrating how difficult it would be to develop the application without these object-oriented techniques.*

Explain the importance of encapsulation, polymorphism and inheritance on object oriented programming

For this task, I will be using examples from the application which I am currently developing to explain the importance of encapsulation, polymorphism and inheritance in the object oriented programming language.

The way this will be done is by first explaining the definition of the object oriented techniques, then showing examples of class diagrams and code in the application, so I can provide the evidence of the definition.

Also providing evidence why it is essential to have this implemented into object oriented programs. By explaining the difficulties that would occur if they were not implemented. With a conclusion for each technique and how they help in the production of the application.

Encapsulation

In the object oriented programming, encapsulation, often referred to as information hiding or data hiding, is basically the process of combining all of an object's methods, properties and attributes of its functionalities into a single class, independently from other classes. Which is an important technique due to the fact it will make the code much safer to execute.

Essentially, with this technique, other classes should not be able to alter an object's attributes. The outside classes should only be allowed to make a request than an attribute should be altered. Hence why it is common to see a class definition to contain set methods and get methods, meaning that it's up to the class's methods to determine whether the request is appropriate.

Below is an example of encapsulation on the application that I am currently developing, to learn about object oriented programming. The class shown in the class diagram and in the form window, for the C# programming language.




```
public class Warhammer : TraumaOrCleavingWeapons
{
    private void Stun()
    {

    }
}
```

As seen in the code above, what is essentially happening is that it is wrapping the “Stun” method within the class alone. This is noticeable since it is private instead of public, which essentially means that any other class, function, variable or method cannot access there directly. Unless, I change the code to be public, or I allow the functions to change the variables themselves inside of that class, or the instance of the class.

So that means, in practice, when I have “Stun” in a class, some other class can use “Stun” and the program will never confuse the two. Due to the fact they are completely different. Which explains the reason it will make the code much safer to develop, because it essentially works by excluding the method from the rest of the program.

To prevent the program getting confused with other similar method, so would be seen as a data integrity feature, by restricting the method only to that class. Hence, without encapsulation, it would increase the chances of the program to get into bugs which seem extremely strange and almost untraceable. Due to the fact that a function or other loops might have the same name as a variable within the code, leading to interference in the code.

Encapsulation can also happen inside of classes themselves. So, if there was a function which used “Stun” inside of it. With another function called “Club”, which also uses “Stun” (with “= 0”). Then, the two functions wouldn’t know that each of the variables existed. This method is called scoping, which means that the “Stun” variable are only available to to function it was set under. Also, there are ways in which they can be overridden, but it is very risky due to the fact that encapsulation contains these variable which shouldn’t be modified by other parts of the code.

However, without encapsulation, this code would not be able to well confined in one place. Meaning that I would have to change all the variables, functions, methods, objects and other forms of code with similar names everywhere. The reason being is that I will be calling the constructor for each and every code within the program.

For example, I would need to modify several files, instead of just one file which contains all the variables on it. Which would cause plenty of issues due to the fact it is more prone to errors and tedious to carry out. Therefore making developing an application unnecessarily difficult to carry out, making some situations almost impossible to deal with.

Overall, encapsulation is essentially hiding attributes within a program. This is so I can make any changes in one place without having to also make changes in the other parts of the application I am developing.

Otherwise, if I didn't have encapsulation available to use for my application, I would make my information more open. Risking a change in one location leading to problems due to cascading, due to other multiple changes found in a different section of the application.

Which means that I would be spending a significant amount of time trying to fix errors, due to variables being confused by the program. On top of this, I would have to also spend time trying to use different names for each variable just incase the program doesn't get confused in the first place, which would cause said errors.

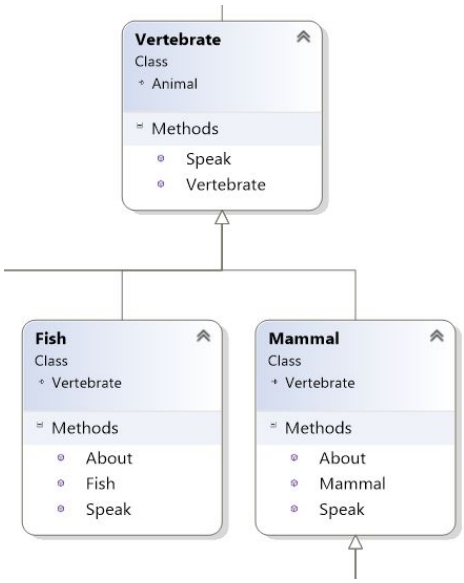
To put it simply, encapsulation is when a group of related methods, properties and other forms of data storage are treated as a single object. It helps separate these variables from being confused by the program, even if they have the exact same name, so the program doesn't get confused.

Polymorphism

In object oriented programming, polymorphism is the ability for an object to be treated and used like another object. Meaning that it is the ability for an object, such as a class or method, to perform in a wide variety of ways. Hence the word, polymorphism, which means of many forms or many-shaped.

Essentially, polymorphism is a very complex but powerful object oriented technique, the most complicated when compared to encapsulation and inheritance. It basically lets the programmer to automatically do the correct behaviour, meaning that what method or object the programmer is working with, it could take one of many different forms.

Below is an example of polymorphism on the application that I have used to learn about object oriented programming. However, it is not one that I am currently creating, the reason being is due to the fact it doesn't have enough features to contain polymorphism on my application's current state. However, in this application, it also shown in the class diagram and form code for the C# programming language.



```
public class Vertebrate : Animal
{
    public Vertebrate()
    {

    }

    public override string Speak()
    {
        return "Some random vertebrate sound";
    }
}
```

```
public class Fish : Vertebrate
{
    public Fish()
    {
        this.Backbone = true;
    }

    public override String Speak()
    {
        return "Gulp gulp";
    }

    public override string About()
    {
        return "I am a fish.";
    }
}
```

```
public class Mammal : Vertebrate
{
    public Mammal()
    {
        this.Backbone = true;
    }

    public override string Speak()
    {
        return "Some generic Mammal noise";
    }

    public override string About()
    {
        return "I am a mammal, so I have fur or hair, feed my young on milk, and I am warm-blooded.";
    }
}
```

Evidently from the screenshots above, polymorphism is the aspect of effectively writing multiple copies of that code, as seen in the “Speak()” and “About()” methods. Each version of the method, will take in a different input.

For example, in the code above, the “Speak()” method will make the “Vertebrate” class return with “Some random vertebrate sound”, the “Fish” class will return with “Gulp gulp” and the “Mammal” method will return with “Some generic Mammal noise”. The same can also be applied to the “About()” methods.

If I were not to have polymorphism, then it wouldn't be possible to to make it so the derived classes have the same methods and method names. Since the program might get confused from the amount of methods with the same name, meaning that there would be an error with the application.

So, this in turn means a reduction in the amount of code required, whilst also keeping the program more uniform and clean for the programmer. Which is very advantageous due to the fact it would be less tedious, stressful and time consuming due to the fact a single method name can be assigned to multiple classes and objects.

Additionally, it would also allow the programmer to then make changes to the base method, in which other classes derive from. Making it easier to debug and make modifications to the program itself. However, it is possible that polymorphism can also possesses its own separate methods.

Overall, polymorphism is writing the same block of code and, most importantly, giving it the same name while also allowing it to take a slightly different type of argument variable, or input. Which by then the program will know if I give the “Fish” class “Speak()”, then the program will know that the program will have the “Fish” class and that it has “Speak()”.

Which it will then use the particular piece of code found within the fish class. In this case, the “Gulp gulp” return method. The same goes for any other method in the program under the “Speak()” or “About()” methods.

Without polymorphism, it wouldn’t be possible to use a single block of code or even change which version of the block of code that is being used, depending on the inputs that it is being given. Meaning that it wouldn’t be possible to assign a single method to multiple classes or objects, increasing clutter and therefore the room for error on the application.

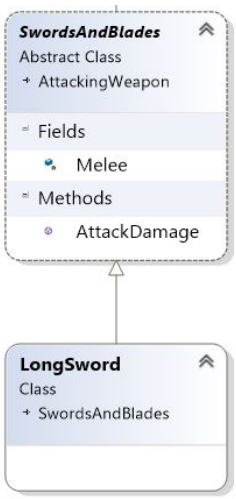
To put it simply, polymorphism is the sharing a common interface for multiple types of classes while having different implementation for different types. Essentially, when each class implements the same methods in varying ways, it still can have several classes that can be utilised, or vice versa.

Inheritance

Inheritance is essentially the process of a class/abstract class acquiring the traits of one’s predecessors, also called a base class. What I mean by this is that the programmer can create or define a new class based on already existing classes. To elaborate, inheritance comes from the fact that the subclass, which is the newly created class, contains the attributes and methods of the parent class.

This can be used to create a highly specialized hierarchical class structure. With the biggest advantage being that there is no need to start from scratch when wanting to specialize an existing class. Basically, inheritance used in object oriented programming refers to the sectioning and re-use of code.

Below is an example of inheritance on the application that I am currently developing, to learn about object oriented programming, shown in the class diagram and form viewer for the C# programming language.




```
public abstract class SwordsAndBlades
{
    private int Melee;

    public void AttackDamage()
    {

    }
}
```

```
public class LongSword : SwordsAndBlades
{
}
```

In the example shown above, it shows the inheritance between a class to two other abstract classes. This is evident from the fact that the screenshot of the bottom code, which is is the “LongSword” class, is inheriting the properties of the “SwordsAndBlades” class, which is visible by the “: SwordsAndBlades” on the public class line.

For “SwordsAndBlades” class, this is the class with the original attributes, which is be known as the parent, super or base class. This parent class is seen containing properties which the “LongSword” class inherited from. With the properties being the “Melee” field and “AttackDamage” method. Additionally, the parent class, which is the “SwordsAndBlades” class, is an abstract class, therefore cannot be used within the program.

The reason that this was done is that it gives me the ability to created an object/class with specific distinguishing features, then possibly reused as a subclass. Which therefore makes the development of the program easier, as it is not required to rewrite code for every object and class. This in turn proves the advantages that inheritance can bring when creating a program with multiple entities.

If I didn’t have the ability to use inheritance with my classes, then there would be a significant amount of repeated code which would increase the file size, decrease the performance / optimisation of the application, making the program more prone to contain an error.

Additionally, if the program did have an error, it would be very difficult to find the said error. Due to the fact there will be an unnecessary amount of repeated code within the program. Which would be time consuming to locate, making it a tedious and stressful task to overcome.

As mentioned before, there are two types of classes in inheritance: The base class, which contains the original code, and the sub-class, which references the parent to inherit functions. Essentially, with the help of inheritance, I was able to use an object, in the form known as a class, to take its attributes from the base class

Overall, inheritance comes from the fact that the subclass contains the attributes and methods of the parent class. Which was extremely helpful in the development with the current development of the application, since I have to write less code to carry out the intended events.

In comparison, if I did not have the availability of inheritance, it would be a lot more time consuming and more prone to causing errors than is necessarily needed. Hence he main advantage of inheritance, which is the ability to define new attributes and new methods for the subclass, which are applied to the inherited attributes and methods.

It is due to this technique of inheritance, that I could use the program to create a basic but specialized hierarchical class structure. With the biggest advantage of this being that there is no need to start from scratch when wanting to specialize an existing class. As a result, class libraries can be created, providing a base that can then be specialized at will by any programmer.

Another feature that occurs with inheritance is that the inherited code will automatically sync with the class it has inherited from. For instance, if I was to change a value of a specific class,

then the inherited on will also have the said changed value, reducing the amount of unnecessary time consumption.

Basically, through inheritance, I was able to create a class, which I then can “inherit” the characteristics of another general class, called a base class. In the case of the program I have created, all “swords and bladed weapons” are for “melee” purposes and can cause “damage “to an “opponent”. This function can be coded into the “SwordsAndBlades” class, meaning that all of its descendants can use it. Therefore reducing the need of repetition.

Conclusion

In order for a programming language to be object-oriented, it must have the ability to work with classes and objects. Therefore, in order to develop an application using the features of an event driven program, it must use the fundamental object-oriented principles of inheritance, polymorphism, and encapsulation.

All of which are vitally important to have. This is due to the fact they can help in the development of an application, while also decreasing the amount of code written in order to produce an application. Which both improves the performance of the application and reduces the file size of the application.

Task 6

(U15:D1) - Evaluate the suitability of object oriented programs for graphical applications

Task description - *Describe, in detail, a good example of an object-oriented program that has a graphical user-interface. Explain clearly why (or why not) Object Oriented techniques are suitable for such GUI based applications/programs. You are permitted to discuss a simple computer game if you wish. Make sure you discuss the key features of OO, and how these have been used.*

Evaluate the suitability of object oriented programs for graphical applications

Throughout extensive research into the suitability of object oriented programs for graphical applications, I have come across how there is a continuous improvement and the increasing spread of graphics hardware for devices and personal computers alike. Which therefore has led to a notable increase to a newer, improved style of software application programs.

Since this constantly improving style relies on fast, high quality graphics displays coupled with expressive input devices, to achieve real-time animation and direct interaction from the user on the programme. Means that such applications bring a significantly challenging and conceptual approach, on both the user and the programmer, than previous software.

Essentially, with this implementation of interactive and graphical interfaces, that allows the visual and the direct control of information in a virtual form, is now an important part of most of modern graphics software tools. Meaning the programs that have to be created have to take in account the amount of performance the graphical interface is using.

However, a good solution to these modern events should result in a system that is highly efficient, reusable and extensible. For instance, the object-oriented paradigm, which is a language in which all relevant information should be encapsulated within objects.

By definition, object-oriented programming is a programming paradigm that essentially revolves everything around the integration of an object. Each of these objects can hold directions and information which can be changed into an object that is used within the program itself. In which it benefits the video game development. Especially in the term of classes, properties, instantiation and other principles of the object oriented paradigm which are essential.

Essentially, there are a great deal of programming dialects which can be used to create an efficient graphical application, such as graphical applications with small to large file capacities. Hence why the example of applications that will be mentioned, which are using the object oriented language, is going to be video games.

To elaborate, in terms of object oriented programming, entities in small videogames are mostly defined by separate classes. However, in larger games they are often defined in files. Typically, there are two general approaches to follow, which are extension and component based.

For extension, the “Troll” and “Werewolf” for example will extend as “Enemy”, which extends into “Entity”. This said, the entity class takes care about the basic things like movement, health and other attributes. While the enemy would state the subclasses as enemies, in order to implement the enemies to do other specific events. Video games such as “Minecraft” follows this approach.

For component based application, the second approach is having just one “Entity” class. Which contains all the components necessary, with a component containing attributes such as “Moveable” or “Enemy”. Essentially, these components take care of one thing like movement or physics.

This method breaks long Extension chains and minimizes the risk of the player of running into a conflict within the application. For example, it wouldn’t be possible for there to be a non-moveable enemy if they inherit from a moveable character.

In well known video games large in file capacity, such as “World of Warcraft” or “Starcraft 2”, “Entities” aren't classes, but sets of data, which specify the whole “Entity” class. Scripting is also important for these type of games, due to the fact it helps define what the entity does, not in a Class, but in a Script. However, only if it is unique, not with generic methods such as the characters moving.

With the composition of how certain video games are made considered. I then researched the different object oriented programming languages that are used for video games, to research their suitability for heavily graphics based applications.

One object oriented language that I have researched and identified to be very good for this situation is C++. Which is one of the many languages that is used for video game development and for other graphical applications. The reason that C++ is utilized generally for GUI's is due to the fact that the language is capable of executing machine level code.

Which is essentially code which is capable of being executed directly by the components of the computer, such as the CPU (Central Processing Unit), increasing the performance capabilities of the language. Also, the high portability of the language and allowing the programmer to have control over much of the program.

The reason stated above is a broad generalisation of what makes C++ a top choice for creating games. However, video games aren't the only benefits for this programming language. Due to the fact it is possible for the programmer to create classes for different segments, which is generally a language capable of being used for a range of applications, most of which will contain a GUI.

Finally, C++ automatically contains event-handling codes, like keyboard events and mouse events which further allows dialogs and menus to be created and changed. Which helps the programmer in the development time for the creation of the program.

Another language that I have researched and identified, that is also used for graphical applications, is Java. Java is also an efficient programming language for graphical applications that are usually based on mobile applications and other software. Usually, Java programs are compiled independently, meaning that it allows the same program to run on any other platform or software that has a Java Virtual Machine installed.

This is a significant advantage for graphical interfaces which are internet based, because it enables the program to run correctly, no matter what system it is running on and as long as the said system has the hardware to support graphical applications. The JVM also has very high security, by limiting access to the program's memory and the system devices which were used.

This is another benefit because the program can be running without affecting the computer's host. Due to the fact that Java is a high-level language, which allows programmers to easily create programs. Therefore making Java a very good language both for beginners and video game developers alike.

The final language that I researched and identified as a graphically efficient object oriented programming language was Python. Python is good for simple graphical applications, such as

“.net” applications. Since it’s a written language, which means that the code is run through an interpreter, rather than being compiled.

Python is also a very robust program, as it has implementations integrated with other languages, such as Jython & PyObjc. Which essentially enables the user to run the application on different languages, therefore making it good for web-based programs.

Also, Python’s ease of use is very well organized, due to the framework which is Django. Essentially, what this framework enables is the ability for the programmer to model their domain, code and website classes. Essentially, this means the programmer can then only focus on the GUI, increasing the quality and optimisation of the graphics and the user interface.

In retrospect, the advantage of using object oriented programming for graphical applications, is that it is easier to maintain and edit existing code. This is an advantage as the programmer is then able to slightly change the previous code to use it in another section, which allows for a more precise program as having to re-write the code again. That has the potential of causing many errors and mistakes within the program.

Another reason why object oriented programming is suitable for graphical applications is because of the use of classes, objects and the hierarchy that the paradigm enables developers to create. To elaborate, it makes more sense to create a software using objects and classes as object oriented programming, since it is based on real world objects with separate entities. Each object has a trait & responsibility, which is like real life entities. Inturn making the program easier to develop by creating modules and re using them as modules.

Furthermore, abstraction is also a benefit when using object oriented programming when developing large graphics applications. Since it enables the programmer to use the significant data, while hiding the irrelevant data on an object. With the use of data abstraction, it will increase the security levels of the program, since the developer can put all their attention to the relevant data and develop those sections. This is good for graphical applications because user information will most likely be inputted so a high level of security, which will support the safety of customer’s information.

This is important due to the fact that graphical applications are usually lasting, because of the high development cost and the focus on user’s needs for additional features, so it is important to focus on this area more.

Yet another benefit that object oriented programming can bring is extensibility. Which makes it able to enhance the system without changing any elements in the system. Instead adding new abilities from a unit which extends another current unit. This can also be achieved through inheritance. So a text window can be updated, to a hyper-text window, by adding more attributes.

With this improvement in hardware and graphics however, the range of applications that benefit from these techniques are getting significantly wider. For example, 2D (two-dimensional) user interfaces popularized by desktop computers like Apple's Macintosh or the Windows computer.

To CAD and animation systems that allow the creation, manipulation and animation of complex 3D (three-dimensional) models, for purposes of scientific visualization of certain experiments or commercial animation for film production.

Unfortunately, the creation and the implementation of such dynamic applications are a complex tasks. Meaning that these systems have to manage an operational model of the real or virtual world, encouraging the evolution of this model in response to events that can occur in an order which is not previously defined.

Essentially, the object oriented programs must be able handle multi-threaded dialogues with the user. Which are essential for direct manipulation interfaces, making extensive use of many asynchronous input devices. Ranging from the common keyboard and mouse, to sophisticated 3D devices such as the Spaceball, Polhemus or DataGlove.

Also, as dynamic graphics applications become larger and more sophisticated, due to the fact they are increasingly moving into the 3D realm with 4K resolution textures, their software engineering problems have become unpleasant. Due to this, it is becoming more evident that software design must make more use of assemblies containing standard components, that can be reused and extended from project to project.

However, in order to build any dynamic graphics application, from a simple 2D user interface manager to a complex 3D animation system, three major problems have to be solved. These three major problems are: The management of the model that the application is supposed to manipulate, the rendering of this model and obtaining the animation/interactive controls, in order to use the application.

While certain object oriented programs make the development of graphical applications, especially video games, possible due to the concept and features of the object oriented paradigm. It still is a problematic obstacle in a development point of view however.

The reason for this is due to the fact that, since interactive and dynamic graphics must respond to real-time input events as they happen, it is usually impossible to know when and in what order the events will come. Therefore, at any moment during the execution of a dynamic application the entire state has to be maintained.

Where most modern object-oriented graphics systems obtain their real-time input in the form of events in an event queue. In this situation, these event queues are generally the preferred method. Although some systems, such as the original Smalltalk, use polling. While others, such as X windows, allow a mixture of the two.

However, even with the benefits of object oriented programming, the challenge of building dynamic graphics applications that realize the full potential of modern computer graphics hardware remains immense. Although, object oriented design techniques provide a significant advance toward the creation of reusable and extensible software components and assemblies for the construction of dynamic graphics.

As object-oriented techniques become more accepted and as language/implementation issues are resolved, more attention can be focussed on the larger design issues. Usually, these principles are able to form the basis for increasingly automated interactive software construction tools, in order to assist the building of the next generation of dynamic graphics applications.

In conclusion, object-oriented programming has many features which make it good for dynamic graphical applications. For instance, it is used on a range of programming languages such as Python, Swift, Java, C++ and, less traditionally in the case of video game production, C#. Due to its features and the method in which the object oriented paradigm functions.

This is beneficial as the programmer has a range of options they can choose from, so they can select the one they have the most knowledge on and is comfortable with. Compared to all the other programming paradigms, object oriented programming specialises in creating GUI elements and is used a lot to develop games, such as the C++ language.

In the production of video games, classes help create a hierarchy of objects found in video games, such as characters and equipment. Through the help of encapsulation, polymorphism and inheritance to create much more refined and simpler to develop classes for video games.

Also, properties help in large graphical applications, due to the fact they can make each class much more detailed. Also with the help of encapsulation, polymorphism and inheritance the classes within the program can be designed to the developer’s content.

Instantiation is also a very useful for large graphical applications, including video games, since it helps with the realisation of predefined objects or classe. Which essentially means that the code of the program can be vastly reduced, making the program much more optimised and easier to debug.

Additionally, another benefit of creating graphical applications such as video games with object-oriented programming include the ability to simulate real-world events much more effectively. Which is done due to the fact that the code can be reusable, thus less code may have to be written, therefore the data becomes active.

Overall, with the features and concept of object oriented programming, programmers are able to reach their goals faster and are also able to produce faster, more accurate and better-written

applications. To put it simply, object-oriented programming provides programmers with many benefits including: maintainability, scalability, and reusability.

Overall Conclusion

In conclusion to both event driven and object oriented programming, the different programming languages can be more efficient depending on the type of application that is intended. Essentially, depending on what the developer is trying to achieve, they could either user event driven or object oriented programming.

For event driven programming, in terms of theory, is essentially the programming paradigm centered in making an application only respond to the user's interaction with the program. With each feature adhering to how a program should be executed through the user's interaction with each event. Essentially, event driven programming is the paradigm of the user's interaction making the program function.

For object oriented programming, in terms of theory, is the programming language paradigm which is essentially organized around the use of objects, instead of actions and data rather than logic. With the use of objects focusing around each other, with these objects being generated in classes in order to define their structure, similar to a hierarchy system or a blueprint. Essentially, object oriented programming is based on objects, which contain data in the form of fields (often seen as attributes), and essential code, in the form of procedures (often seen as methods).

Overall, each programming language in useful in their own respect, but it depends on the type of program they are being designed for. Since both object oriented and event driven languages are completely different in structure, in order to adhere to the use of the wide arrays of applications within a computer system.

For instance, event driven programs would prove to be most useful if it's being made for an application which completely relies on the user's input in order to function properly, such as an operating system. While object oriented programming may be used for an application which is centered around programs which relies on a hierarchical system or it contains many different entities which require different abilities from each other, such as a database application.

However, there are applications that use both event driven and object oriented paradigms, such as video games. The reason being is that applications like this both require complete user interaction in order to complete its purpose. Also, the use of different entities, which contain different abilities and purposes from each other. Basically, object oriented and event driven programming have a wide array of uses within the computer system world, and are integral to make it possible for use depending on the type of use.